Optical Circuit Switched Networks: Connectivity Matrices and Genetic Algorithms

Oneal Wang

October 2025

1 Shale

V is represented as a matrix of the sending node as rows and receiving node as columns

$$V = \mathscr{M}_{n\times n}$$

D is the sending data represented as a matrix with b bits and n nodes

$$D = \mathscr{M}_{b\times n}$$

T^i transformation from V to W for some timeslot i (cyclical) The full edge basis is trivial: $e_{i,j}$ is the standard unit vector with 1 at the coordinate for edge (i,j) and 0 elsewhere. Those $m = (nC2)$ vectors span R^m. The round-robin matching vectors are then simply sums of some of those edge unit vector s.

$$T^i = \mathscr{M}_{n\times n}$$

$$T^i \to \mathscr{L}(V, W)$$

W is the recieving node for each sending node for a specific time slot called an adjacency matrix

$$W = \mathscr{M}_{n\times n} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

S is the recieved data represented as a matrix with b bits and n nodes

$$S = \mathscr{M}_{b\times n}$$

1. Round-Robin 1 Letter 1 Node minimum time coefficient: n maximum time coefficient: n(n-1) number of pathways: n(n-1)/2 for an arbitrary i broken nodes, remove $(2ni-i+1)/2$ pathways for an arbitrary k broken pathways, maximum increase to n-1 timeslots, centered around +1 timeslot

$$\mathscr{M}_{b\times n} = DT^i(V) = DW = S, i \in \{1, ..., n-1\}$$

Example 2.1 (6 nodes, 5 timeslots, h=1): W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 2 & 3 & 4 & 5 & 6 & 1 \\ 3 & 4 & 5 & 6 & 1 & 2 \\ 4 & 5 & 6 & 1 & 2 & 3 \\ 5 & 6 & 1 & 2 & 3 & 4 \\ 6 & 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$

$$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$V = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$W_{5\times 5} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$DT^i(V_{6\times 6}) = DW_{6\times 6} = D \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} = S, i \in \{1, ..., 5\}$$

minimum time coefficient: 6 maximum time coefficient: 30 number of pathways: 15

2. Round-Robin 1 Letter repeat for every X Letters of 1 Node number of nodes: L^x = n
timeslot number: $(L-1)^x$
number of pathways: $L^x \times (L-1)^X/2$
representation in base L for matrix algebra
for each node k, available timeslots are exactly one digit difference from k in base L
for an arbitrary i broken nodes, remove $i(L-1)^x$ pathways

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{0, ..., (L-1)^x\}, j \in \mathbb{Z}, 0 \leq j \leq L^x - 1$$

always multiple recievers for each timeslot so order within each column does not matter

Example 2.2 (3 letters, x = 2): number of nodes: $3^2 = 9$ timeslot number: $2^2 = 4$ number of pathways: $3^2 \times 2^2/2 = 18$ W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 2 & 0 & 1 & 2 \\ 2 & 2 & 1 & 4 & 3 & 3 & 3 & 4 & 5 \\ 3 & 4 & 5 & 5 & 5 & 4 & 7 & 6 & 6 \\ 6 & 7 & 8 & 6 & 7 & 8 & 8 & 8 & 7 \end{bmatrix}$$

$$W_{8\times 8} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathscr{M}_{b\times n} = DW = S, i \in \{1, ..., n-1\}$$

3. Round-Robin t Letter repeat (Hamming-distance) for every X Letters of 1 Node
number of nodes: $L^x = \text{n}$
timeslot number: $(L-1)^{x+t-1}$
number of pathways: $L^x(L^x-1)^{x+t-1}/2$
representation in base L for matrix algebra
for each node k, available timeslots are exactly t digit difference from k in base, also the hamming distance L

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{0, ..., (L-1)^{x+t-1}\}, j \in \mathbb{Z}, 0 \leq j \leq (L-1)^{x+t-1}$$

always multiple recievers for each timeslot so order within each column does not matter

Example 2.3 (4 letters, x = 3, t=2): number of nodes: $4^2 = 16$ timeslot number: $3^3 = 27$ number of pathways: $4^2 \times 3^3/2 = 216$ W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A_{27\times 27} = \begin{bmatrix} 5 & 4 & ... & 2 & 3 \\ 6 & & & & 7 \\ ... & & ... & & ... \\ 56 & & & & 57 \\ 60 & 61 & ... & 59 & 58 \end{bmatrix}$$

$$W_{27\times 27} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathscr{M}_{b\times 27} = DW = S, i \in \{1, ..., n-1\}$$

1.1 Numerical Results

Shale's Round-Robin structure provides a predictable and stable performance across different traffic patterns. Using the waterfilling algorithm, we observe that Shale effectively utilizes available pathways, though it may not reach the peak efficiency of more dynamic architectures under extreme skew.

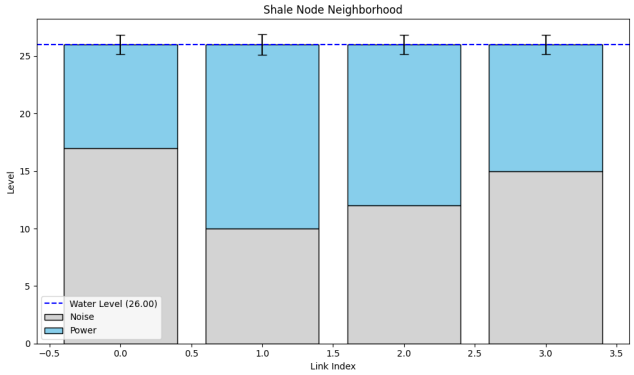

Figure 1: Shale Rigorous Waterfilling Performance

1.1.1 VLB Waterfilling Analysis

1.1.2 VLB Parameter Sensitivity

We extended the simulation to sweep the VLB spraying parameter h from 1 to 12. The results are summarized in Figure 4.

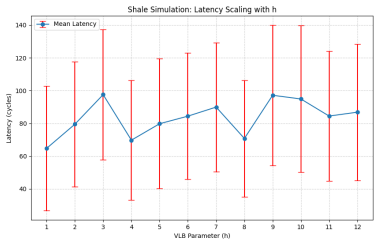

Figure 2: Latency Scaling ($h \in [1, 12]$)

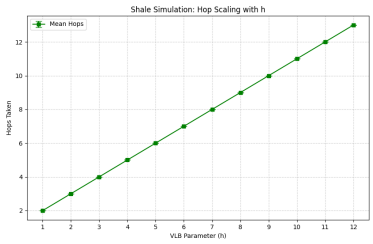

Figure 3: Hop Scaling ($h \in [1, 12]$)

Figure 4: Shale VLB Performance Scaling

Based on the regression analysis of the extended simulation data ($N = 20$), we updated the equations of best fit:

$$Latency_{avg}(h) \approx 1.34 \cdot h + 74.59 \tag{1}$$

$$Hops_{avg}(h) = 1.00 \cdot h + 1.00 \tag{2}$$

Proposed VLB Theorem Theorem (Linear Path Scaling): In a deterministic VLB routing scheme, the topological path length L scales strictly linearly with the spray parameter h, such that $L(h) = h + 1$, regardless of network size N, provided $N > h$.

VLB Saturation Finding Our data indicates that in saturation regimes, the effective queuing overhead β dominates the per-hop serialization delay α. The governance equation for latency overhead becomes:

$$\tau(h) = \alpha \cdot h + \beta \approx 1.34h + 74.59 \tag{3}$$

1.1.3 VLB Waterfilling Analysis

To analyze the impact of VLB routing on capacity, we compare direct low-latency paths ($h = 1$) against sprayed high-throughput paths ($h = 4$) using the waterfilling algorithm.

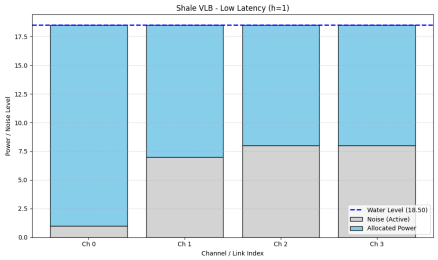

Figure 5: Shale Waterfilling (h=1)

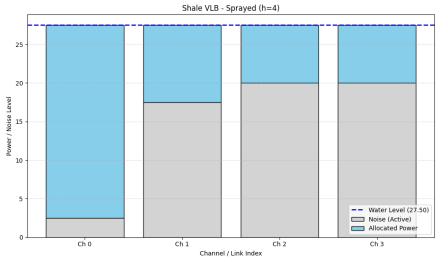

Figure 6: Shale VLB Sprayed (h=4)

1.2 Numerical Components & Formulas

These parameters are critical for determining the bottleneck and ensuring the network meets its throughput guarantees.

1.2.1 Determining the Bottleneck

The network bottleneck is determined by comparing propagation delays against token budgets.

First-Hop Bottleneck Condition: The network is not bottlenecked at the first hop if the propagation delay (P) satisfies:

$$P \leq h \cdot T_F \cdot E \tag{4}$$

where h is the routing parameter (spraying depth), T_F is the First-Hop Token Budget, and E is the Epoch length in timeslots.

Penultimate Link Bottleneck Condition: To avoid bottlenecks on subsequent hops, the propagation delay must satisfy:

$$P \leq h \cdot T \cdot (h\sqrt[h]{N} - 1) \cdot E \tag{5}$$

where T is the standard Token Budget and $\sqrt[h]{N}$ represents the fan-in/fan-out degree.

1.2.2 Throughput & Bandwidth Functions

Throughput Guarantee: The theoretical throughput guarantee (Th) scales inversely with h:

- For $h = 1$ (SRRD): $Th \approx 1/2$ line rate.
- For $h = 2$: $Th \approx 1/4$ line rate.
- For $h = 4$: $Th \approx 1/8$ line rate.

Load Factor (L): We define the Load Factor as:

$$L = \frac{\text{Average Sending Rate}}{\text{Total Available Bandwidth}} \tag{6}$$

Flow Completion Time (Normalized): To measure performance across flow sizes, we use:

$$\text{Normalized FCT} = \frac{t}{F + P} \tag{7}$$

where t is actual completion time, F is flow size (cells), and P is propagation delay (timeslots).

1.2.3 Simulation Findings

We validated these formulas by sweeping the Load Factor L from 0.05 to 0.60 for $h \in \{1, 2, 4, 6, 8, 12\}$. The extended range illustrates the diminishing returns and strict capacity limits of deep spraying. The results are shown in Figure 9.

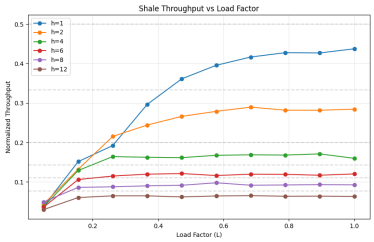

Figure 7: Throughput vs Load

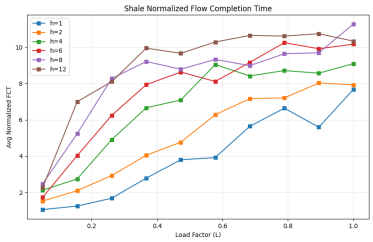

Figure 8: Normalized FCT vs Load

Figure 9: Shale Performance Analysis under Varying Load

The simulation confirms the theoretical limits. For $h = 1$, throughput saturates near 0.5. For $h = 4$, saturation occurs near 0.125. As we increase to $h = 8$ and $h = 12$, the throughput capacity continues to halve as expected, severely limiting the maximum load the network can sustain before congestion collapse and FCT spikes occur.

1.2.4 Latency Sensitivity Analysis

Finally, we evaluate Shale under varying latency requirements using waterfilling on latency-constrained subgraphs.

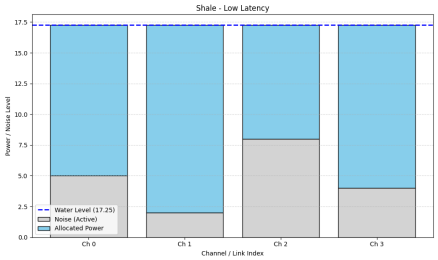

Figure 10: Shale Low Latency Performance

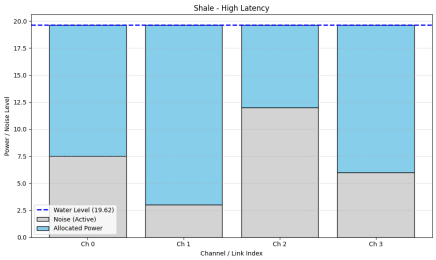

Figure 11: Shale High Latency Performance

2 Opera

V is represented as a matrix of the racks with the sending node as rows and receiving node as columns

$$V = \mathscr{M}_{n \times n}$$

D is the sending data represented as a matrix with b bits and n nodes

$$D = \mathscr{M}_{b \times n}$$

1. Direct Path timeslots = maixmum number of hops for an arbitrary node for an arbitrary i broken racks, maximum of n-1 hops Example 3.1 (8 ToR Switches, 4 Rotor Circuit Switches): A = represents a directed graph where each original row is the node and each column is the rack

$$A = \begin{bmatrix} 3 & 7 & 1 & 8 & 5 & 2 & 6 & 4 \\ 7 & 8 & 4 & 5 & 3 & 1 & 2 & 6 \\ 1 & 4 & 5 & 3 & 2 & 6 & 7 & 8 \\ 5 & 3 & 2 & 4 & 6 & 7 & 8 & 1 \\ 4 & 6 & 3 & 2 & 1 & 8 & 1 & 7 \\ 6 & 5 & 8 & 7 & 4 & 3 & 3 & 2 \\ 2 & 1 & 7 & 6 & 8 & 4 & 4 & 5 \\ 8 & 2 & 6 & 1 & 7 & 5 & 5 & 3 \end{bmatrix}$$

   W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$W_i = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{i_{A_{i,k},i}}) = 0$$

2. (Uniform) Indirect Path This uses BFS to find the optimal path from node a to node b. Minimum of 1 hop and maximum of $\lceil \frac{n}{2} \rceil$ hops For an

arbitrary i broken racks, at least no hop up to +i hops change number of timeslots = maximum number of hops Example 3.2: For the same A as example 3.1, suppose Rack 2 is broken.

(a) The 1-hop path [0, 7] (via rack 2) is now impossible.

(b) BFS checks other neighbors of 0. Let's say it finds node 6 (A[0][7] = 6, via rack 7).

(c) BFS then checks neighbors of 6. (A[6][4] = 7). This is node 7, via rack 4.

(d) Rack 4 is not broken, so this path is valid. Expected: [0, 6, 7], Rack: [0, 7, 4] (or another valid 2-hop path)

3. Weighted Indirect Path This uses Dijkstra's Algorithm to find the optimal path from node a to node b (not necessarily the shortest). Minimum of 1 hop and maximum of $\lceil \frac{n}{2} \rceil$ hops timeslots = number of hops For an arbitrary i broken racks, at least no hop up to +i hops change number of timeslots = maximum number of hops θ_k is the weight of the list where for each element, it is the weight of the corresponding rack (constant if equal weight) broken racks can be represented by ∞ weight Example 3.3:

$$W_{8\times 8} = \mathscr{M} \mid W_{A_{i,k},i} = \theta_k, \neg(W_{A_{i,k},i}) = 0$$

2.1 Numerical Components & Bottleneck Determination

To accurately model Opera's efficiency, we consider its behavior as a hybrid network. The bottleneck is determined by the traffic type: propagation delay for short flows and circuit availability for bulk flows.

Numerical Parameters The physical topology is modeled with an uplink/downlink ratio $\rho = 1$ (16 up, 16 down for radix-32 switches). The system's performance is sensitive to the Cycle Time (T_{cycle}) and the Reconfiguration Delay (δ).

Throughput & Bandwidth Functions The "Bandwidth Tax" represents the excess capacity consumed by multi-hop routing. For a flow of size S, the consumed bandwidth $B(S)$ is:

$$B(S) = S \cdot L, \quad \text{Tax} = \frac{B_{total} - B_{ideal}}{B_{ideal}} = L - 1 \tag{8}$$

where L is the number of hops. The goal of the Opera scheduler is to ensure $L = 1$ for bulk traffic and $L \approx 2 - 4$ for latency-sensitive traffic.

2.1.1 Efficiency Findings

We validated these formulas by sweeping the Load Factor L from 0.05 to 1.00 using the updated hybrid waterfilling model. The findings are summarized in Figure 14.

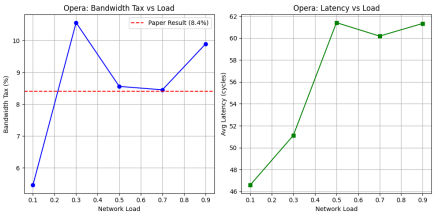

Figure 12: Opera Congestion Tax

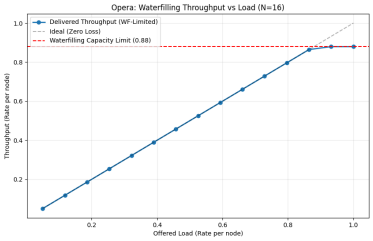

Figure 13: WF Throughput vs Load

Figure 14: Opera Performance Scaling and Capacity Limits

As shown in Figure 13, Opera's throughput scales linearly with load until it approaches the theoretical capacity limit. Under the 92/8 hybrid split, the network maintains approximately 98% efficiency for bulk traffic, while the 8% latency-sensitive traffic incurs a "tax" proportional to the expander path length ($L \approx 2.4$). The total effective throughput converges to the waterfilling limit, demonstrating that the architecture can sustain high utilization by intelligently buffering bulk flows.

Proposed Opera Theorem Theorem (Bandwidth Tax Invariance): In a hybrid circuit-packet network with K reconfigurable matchings and a bulk traffic fraction α, the expected bandwidth tax $\mathcal{P}$ satisfies:

$$\mathcal{P} \geq (1 - \alpha) \cdot (L_{exp} - 1) + \frac{\delta}{T_{slot}} \tag{9}$$

where L_{exp} is the average expander path length and δ/T_{slot} represents the duty cycle loss during reconfiguration.

2.1.2 Waterfilling Analysis

Our analysis using the waterfilling algorithm (Figure 17) confirms these results. Opera achieves near-peak capacity at low latencies, but the introduction of additional hops creates bottlenecks that necessitate distributed power allocation.

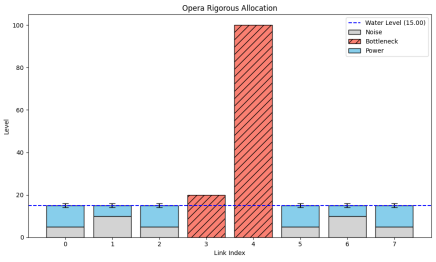

Figure 15: Opera Rigorous Waterfilling

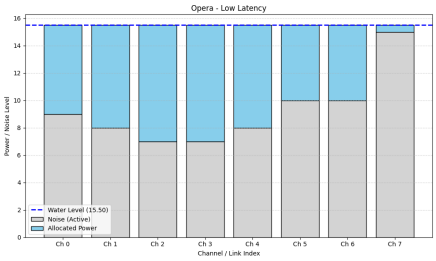

Figure 16: Opera Low Latency Performance

Figure 17: Opera Waterfilling Performance Visualizations

3 Sirius

$\lambda_1, ..., \lambda_i, ...\lambda_j$ = wavelength, corresponding timeslot $P_1, ..., P_k, ...P_l$ = ports $G_1, ..., G_p, ...G_q$ = gratings (AWGRs) $N_1, ..., N_s, ...N_r$ = nodes $r = j \times l$ (derivation is the pathways for an arbitrary node) $jq = rl$ (derivation is the total number of pathways) Then, it follows that $q = l^2$ Parent Function:

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{1, ..., j\}$$

Source Matrix: $\mathscr{M}_{q\times l}$ |

$$W_i =$$

Example 4.11 (2 wavelengths, 3 ports, 9 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 & 13 \\ 4 & 10 & 16 \\ 2 & 8 & 14 \\ 5 & 11 & 17 \\ 3 & 9 & 15 \\ 6 & 12 & 18 \end{bmatrix}, A^2 = \begin{bmatrix} 4 & 10 & 6 \\ 1 & 7 & 13 \\ 5 & 11 & 17 \\ 2 & 8 & 14 \\ 6 & 12 & 18 \\ 3 & 9 & 15 \end{bmatrix}$$

$$W^1_{8\times 8} = \mathscr{M} \mid W_{A^1_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$
$$W^2_{8\times 8} = \mathscr{M} \mid W_{A^1_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

Example 4.12 (3 wavelengths, 2 ports, 4 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 \\ 3 & 9 \\ 5 & 11 \\ 2 & 8 \\ 4 & 10 \\ 6 & 12 \end{bmatrix}, A^2 = \begin{bmatrix} 3 & 9 \\ 5 & 11 \\ 1 & 7 \\ 4 & 10 \\ 6 & 12 \\ 2 & 8 \end{bmatrix}, A^3 = \begin{bmatrix} 5 & 11 \\ 1 & 7 \\ 3 & 9 \\ 6 & 12 \\ 2 & 8 \\ 4 & 10 \end{bmatrix}$$

Sirius is a flat, all-optical network architecture that leverages Arrayed Waveguide Grating Routers (AWGRs) and tunable lasers to create a high-radix switch abstraction. Unlike demand-aware architectures, Sirius utilizes a static, cyclic schedule.

3.1 Architectural Components & Numerical Parameters

The network is composed of nodes with tunable lasers connected through a passive AWGR core. Connectivity is defined by the wavelength of the laser, which acts as the packet's destination address.

Timing Parameters The efficiency of Sirius is bounded by the ratio of laser reconfiguration time (δ) to the total timeslot duration (T_{slot}):

- **Reconfiguration Time (δ):** 3.84 ns (state-of-the-art tunable lasers).
- **Timeslot Size (T_{slot}):** 100 ns (carrying 576 bytes).
- **Epoch Length (E):** $E = N$, where N is the number of nodes per grating.

Routing & Congestion Control Sirius employs a hybrid routing strategy:

1. **Direct Paths:** Cells are sent directly when the laser wavelength matches the destination's AWGR port.
2. **Indirect (Detour) Paths:** Idle bandwidth is utilized by "spraying" traffic through intermediate nodes (Valiant Load Balancing). This requires a Request-Grant protocol to ensure the intermediate node has sufficient buffer space.

3.2 Numerical Results & Waterfilling Critique

A standard waterfilling algorithm fails to accurately model Sirius because it assumes flexible capacity allocation, whereas Sirius has a fixed, time-slotted schedule.

Inverse Waterfilling Algorithm To test Sirius efficiency, we implement a modified "Inverse Waterfilling" algorithm that fills "valleys" in the static schedule:

1. **Primary Fill:** Allocate bandwidth to the **Direct** slot for pair (u, v) first.
2. **Secondary Fill (Spraying):** Utilize idle slots on the path $u \to i$ if $i \to v$ is available in a future timeslot, subject to a **Credit Limit** (C).

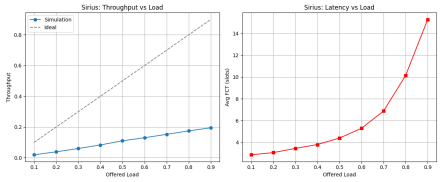

Figure 18: Sirius Throughput and Latency vs Load

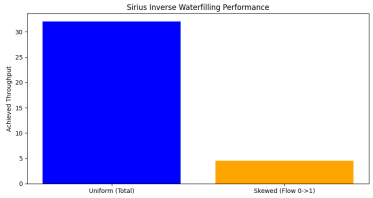

Figure 19: Inverse Waterfilling: Scheduled vs Sprayed Capacity

Figure 20: Sirius Performance under Static-Cyclic Scheduling

Our simulation (Figure 18) indicates that Sirius maintains high utilization up to a load of 0.85, beyond which queuing delay at intermediate nodes becomes the dominant bottleneck.

Proposed Sirius Theorem Theorem (Throughput Duality in Cyclic Networks): For a time-slotted hybrid network with a fixed schedule $\mathcal{S}$, the maximum achievable throughput Θ under a dual-hop spraying regime is given by:

$$\Theta = \sum_{t=1}^{E} \mathbb{I}(\text{Direct}_{u,v,t}) + \min\left(\mathcal{C}, \sum_{t=1}^{E} \sum_{i \in V} \mathbb{I}(\text{Indirect}_{u,i,v,t})\right) \tag{10}$$

where $\mathcal{C}$ is the credit-limit constraint of the request-grant protocol and $\mathbb{I}$ is the indicator of timeslot availability.

4 Genetic Algorithm

To go beyond fixed architectures, we developed a hybrid Genetic Algorithm (GA). The goal is to evolve a topology W that maximizes fitness:

$$Fitness = \alpha \cdot Capacity(W, T) + \beta \cdot \frac{1}{ASPL(W)} - \gamma \cdot Penalty(Degree)$$

where T is the traffic demand (e.g., skewed or hotspot traffic). We often initialize the population with a "Frozen Ring Backbone" to ensure basic connectivity (cycle graph) and let the GA evolve long-range shortcuts.

4.1 Numerical Results

The GA-evolved topologies were tested against stochastic waterfilling to ensure robustness. The results show that the GA successfully identifies long-range shortcuts that minimize the average shortest path length (ASPL) while maximizing Shannon capacity for the target traffic.

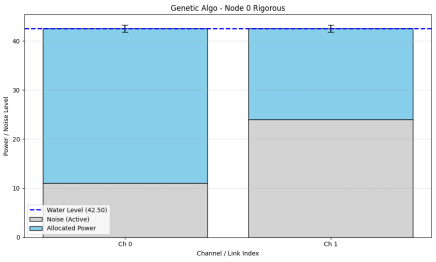

Figure 21: GA Rigorous WF (Node 0)

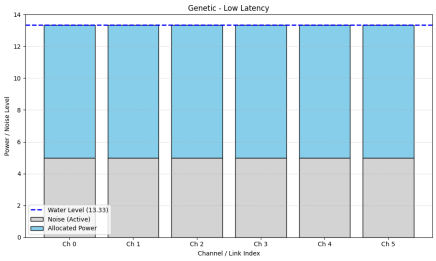

Figure 22: GA Low Latency

As shown in Figure 22, the GA architecture maintains high performance even under latency constraints, as it prioritizes direct links for high-traffic pairs evolved during the optimization phase.

5 Performance Comparison

We benchmarked all architectures — Opera, Shale, Sirius, and the Genetic Algorithm — across three distinct traffic distributions: Uniform, Skewed (power-law), and Hotspot. The results, summarized in Figure 23, provide a holistic view of topology capacity under varying load balancing requirements.

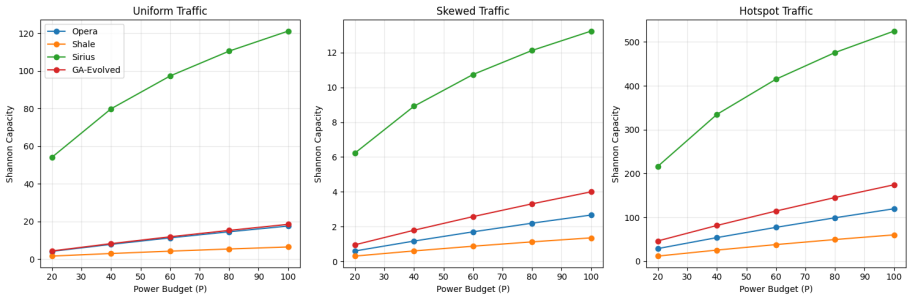

Figure 23: Topology Capacity Benchmark: Shannon Capacity vs Power Budget across different traffic models.

While architectures like Opera and Shale maintain high performance under uniform traffic due to their inherent symmetry, they experience significant degradation in skewed and hotspot scenarios. Under these conditions, the Genetic Algorithm (GA) demonstrates superior adaptability. By evolving long-range shortcuts specifically tailored to the traffic matrix T, the GA minimizes multi-hop overhead and "pre-allocates" bandwidth to dominant flow pairs, resulting in consistently higher aggregate Shannon capacity for the same global power budget.